AMRITA VISHWA VIDYAPEETHAM

श्रद्धावान् लभते ज्ञानम्

NAME: RAAGHAV VEL P

CSE-B

CH.SC.U4CSE24155

1.Write a C program to implement Huffman Coding.

- Take a string as input
- Count the frequency of each character
- Generate Huffman codes for all unique characters

Input String:
"DATAANALYTICSANDINTELLIGENCELABORATORY"

Code:

```c
//CH.SC.U4CSE24155
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 256

// Node structure
struct Node {
    char data;
    int freq;
    struct Node *left, *right;
};

// Min Heap structure
struct MinHeap {
    int size;
    struct Node* array[MAX];
};

// Create a new node
struct Node* createNode(char data, int freq) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->data = data;
    temp->freq = freq;
    temp->left = temp->right = NULL;
    return temp;
}

// Swap two nodes
void swap(struct Node** a, struct Node** b) {
    struct Node* t = *a;
    *a = *b;
    *b = t;
}
```

```c
// Heapify
void heapify(struct MinHeap* heap, int i) {
    int smallest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if (left < heap->size && heap->array[left]->freq < heap->array[smallest]->freq)
        smallest = left;

    if (right < heap->size && heap->array[right]->freq < heap->array[smallest]->freq)
        smallest = right;

    if (smallest != i) {
        swap(&heap->array[smallest], &heap->array[i]);
        heapify(heap, smallest);
    }
}

// Extract min
struct Node* extractMin(struct MinHeap* heap) {
    struct Node* temp = heap->array[0];
    heap->array[0] = heap->array[heap->size - 1];
    heap->size--;
    heapify(heap, 0);
    return temp;
}

// Insert node
void insertHeap(struct MinHeap* heap, struct Node* node) {
    heap->size++;
    int i = heap->size - 1;

    while (i && node->freq < heap->array[(i - 1) / 2]->freq) {
        heap->array[i] = heap->array[(i - 1) / 2];
        i = (i - 1) / 2;
    }
    heap->array[i] = node;
}
```

```c
// Build Huffman Tree
struct Node* buildHuffman(char data[], int freq[], int size) {
    struct MinHeap heap;
    heap.size = 0;

    for (int i = 0; i < size; i++)
        heap.array[i] = createNode(data[i], freq[i]);
    heap.size = size;

    // Build initial heap
    for (int i = (heap.size - 1) / 2; i >= 0; i--)
        heapify(&heap, i);

    while (heap.size > 1) {
        struct Node* left = extractMin(&heap);
        struct Node* right = extractMin(&heap);

        struct Node* top = createNode('$', left->freq + right->freq);
        top->left = left;
        top->right = right;

        insertHeap(&heap, top);
    }

    return extractMin(&heap);
}

// Print Huffman Codes
void printCodes(struct Node* root, int arr[], int top) {
    if (root->left) {
        arr[top] = 0;
        printCodes(root->left, arr, top + 1);
    }

    if (root->right) {
        arr[top] = 1;
        printCodes(root->right, arr, top + 1);
    }
```

```c
    if (!root->left && !root->right) {
        printf("%c\t%d\t", root->data, root->freq);
        for (int i = 0; i < top; i++)
            printf("%d", arr[i]);
        printf("\n");
    }
}

void main() {
    char str[] = "DATAANALYTICSANDINTELLIGENCELABORATORY";

    int freq[MAX] = {0};

    // Count frequency
    for (int i = 0; str[i]; i++)
        freq[(int)str[i]]++;

    char data[MAX];
    int frequency[MAX];
    int size = 0;

    // Store unique characters
    for (int i = 0; i < MAX; i++) {
        if (freq[i] > 0) {
            data[size] = (char)i;
            frequency[size] = freq[i];
            size++;
        }
    }

    // Build Huffman Tree
    struct Node* root = buildHuffman(data, frequency, size);

    int arr[MAX], top = 0;

    printf("Char\tFreq\tHuffman Code\n");
    printCodes(root, arr, top);
}
```

Output:

```
C:\II year\4th sem\DAA\LAB\week-8\prog>.\a
Char    Freq    Huffman Code
N       4       000
L       4       001
O       2       0100
R       2       0101
C       2       0110
B       1       01110
G       1       01111
T       4       100
Y       2       1010
S       1       10110
D       2       10111
E       3       1100
I       3       1101
A       7       111
```

Time Complexity

Frequency calculation: $O(n)$

Building priority queue (min heap): $O(n)$

Building Huffman Tree: $O(n\log\ n)$

Overall Time Complexity:

$$O(n\log\ n)$$

Space Complexity

Frequency array / map: $O(n)$

Huffman Tree nodes: $O(n)$

Overall Space Complexity:

$$O(n)$$